

Voicecon Mobile App — Chat Inbox

API guide for the Flutter app: sign-in, chatbots, conversation inbox, search, team replies, live updates and push notifications

Version 1.0 · 7 October 2026 · Backend: Voicecon API v1

Contents

1. What the app does
2. Basics: URLs, headers, formats, errors
3. What you need from the Voicecon team
4. Sign-in and sessions
5. Workspaces
6. Chatbots
7. Inbox: conversations, search, filters
8. One conversation: messages, reply, takeover
9. Live updates (WebSocket)
10. Push notifications
11. Putting it together: recommended app flow
12. Suggested Flutter packages
13. Endpoint quick reference
14. Test checklist

1. What the app does

Voicecon customers build **chatbots** in the web app (app.voicecon.ai) and put them on their websites. Each chatbot is answered by an AI agent. The mobile app is the customer's **inbox** for those website chats. Customers keep creating and editing chatbots on the web; the app is for following and answering conversations.

- **Sign in** with the same account as the web app (email + password, Google, or Apple).
- **See every chatbot** in the account. One account can have **many chatbots** (for example "Main website", "Online shop", "Help centre"). The app must list all of them, never assume there is only one.
- **Inbox** of all conversations across all chatbots, newest activity first, with unread badges. Filter by one chatbot.
- **Search** conversation history (searches message text, the page URL and the visitor id).
- **Open a conversation** to read the full transcript (visitor, AI agent and team messages).
- **Reply** to the visitor from the phone. Replying takes the conversation over from the AI ("human mode"). A "Hand back to AI" button returns it to the agent.
- **Push notification** when a visitor starts a chat or sends a message, even when the app is closed. Tapping it opens that conversation.
- **Live updates** while the app is open: new messages appear without pull-to-refresh.

Suggested screens

Screen	Content	APIs
Sign in	Email + password, "Continue with Google", "Sign in with Apple" (iOS), Forgot password	<code>/auth/login</code> , <code>/auth/google</code> , <code>/auth/apple</code> , <code>/auth/password/*</code>

Workspace picker	Only if the user belongs to more than one workspace. Can live in a settings menu.	GET <code>/workspaces</code>
Inbox (home)	All conversations. Chatbot filter chips, search bar, "Unread" / "Needs reply" filters, unread badges.	GET <code>/chat/sessions</code> , GET <code>/chat/chatbots</code>
Chatbots	List of all chatbots with live status and unread count. Tap opens the inbox filtered to that chatbot.	GET <code>/chat/chatbots</code>
Conversation	Header (visitor, chatbot, AI/Team mode), message bubbles, reply box, "Take over" / "Hand back to AI" action.	GET <code>/chat/sessions/{id}</code> , <code>.../messages</code> , <code>.../read</code> , <code>.../takeover</code> , <code>.../release</code>
Settings	Account, workspace switch, notifications on/off, sign out.	GET <code>/users/me</code> , <code>/devices</code>

2. Basics: URLs, headers, formats, errors

Item	Value
REST base URL (production)	<code>https://api.voicecon.ai/api/v1</code>
WebSocket URL (production)	<code>wss://api.voicecon.ai/api/v1/chat/ws</code>
Local/dev server	Ask the Voicecon team for a staging URL; the paths are identical.
Format	JSON request and response bodies, UTF-8. Send <code>Content-Type: application/json</code> .
Auth header	<code>Authorization: Bearer <access_token></code> on every call except sign-in.
Workspace header	<code>X-Organization-Id: <workspace id></code> on every <code>/chat/*</code> call (see section 5).
IDs	All ids are UUID strings.
Timestamps	ISO 8601 in UTC with offset, e.g. <code>2026-10-07T13:28:50.526107+00:00</code> . Parse with <code>DateTime.parse()</code> and call <code>.toLocal()</code> to display.
Paging	<code>page</code> (from 1) and <code>page_size</code> (max 100). Responses include <code>total</code> and <code>has_more</code> .
Rate limits	About 300 reads and 60 writes per minute per user per endpoint. A <code>429</code> means wait and retry.

Errors

Errors return a JSON body with a human-readable `detail` string that is safe to show to the user. Validation errors (`422`) use a different shape.

```
// 401, 403, 404, 400 ...
{ "detail": "Session not found" }

// 401 when the account was deactivated or deleted
{ "detail": "Your account has been deactivated or deleted. ...", "code": "account_inactive" }

// 422 validation error
{ "error": "ValidationError", "message": "Request validation failed",
  "details": [ { "loc": ["body", "content"], "msg": "String should have at least 1 character" } ]
}
```

Status	Meaning	What the app should do
401	Access token missing, expired or revoked	Call <code>/auth/refresh</code> once and retry. If refresh also fails, go to the sign-in screen. If <code>code == "account_inactive"</code> , show that message.
402	The workspace's plan or trial has ended	Reading still works. Sending replies and other writes are blocked. Show "Your plan has ended — manage billing on the web app".
403	The user's role does not allow this. Viewers can read but cannot reply, take over or mark read.	Hide or disable the reply box for that workspace (see role in <code>GET /workspaces</code>).
404	Not found, or not in this workspace	Go back to the list and refresh it.
422	Invalid request body	A bug in the request; log it.
429	Too many requests	Back off for a few seconds and retry.
5xx	Server problem	Show a generic "Something went wrong, try again".

Roles

Each workspace membership has a role: `owner`, `admin`, `member` or `viewer`. Owner, admin and member can do everything in this guide. **Viewer** can read chatbots and conversations and receives push notifications, but gets `403` on reply, takeover, release and mark-read.

3. What you need from the Voicecon team

Item	Why
Firebase project access, plus <code>google-services.json</code> (Android) and <code>GoogleService-Info.plist</code> (iOS)	Push notifications. The Voicecon team owns the Firebase project and puts its service-account key on the server. The app and the server must use the same Firebase project.
Android package name and iOS bundle id to use (e.g. <code>ai.voicecon.app</code>)	Registered in Firebase, Google Cloud and Apple. The server must accept the iOS bundle id for Apple sign-in (<code>APPLE_BUNDLE_IDS</code>).
Google OAuth client ids (Android, iOS) in the Voicecon Google Cloud project	Google sign-in. Your Android signing SHA-1 / SHA-256 must be added to the Android client.
APNs key uploaded to Firebase	iOS push goes through Firebase → APNs.

A staging API URL and a test account with two or more chatbots	Development and QA.
--	---------------------

There are **no webhooks** for the app to receive. The server pushes events to the app in two ways: **Firestore push** (app in background or closed) and the **WebSocket** (app open). Everything else is plain REST.

4. Sign-in and sessions

Every sign-in method returns the same response: an **access token** (valid about 7 days) and a **refresh token** (valid about 30 days). Store both in secure storage (Keychain / Keystore via `flutter_secure_storage`), never in plain shared preferences.

```
{
  "access_token": "eyJhbGciOi...",
  "refresh_token": "eyJhbGciOi...",
  "token_type": "bearer",
  "user": {
    "id": "ca34a5b7-147a-4875-a154-28ce0c2a8feb",
    "email": "sara@example.com",
    "full_name": "Sara Khan",
    "avatar_url": null,
    "is_verified": true,
    "auth_provider": "email",
    "is_new": false
  }
}
```

POST `/auth/login` — email and password

```
{ "email": "sara@example.com", "password": "....." }
```

Wrong email or password → `401`. After repeated failures the address is locked for a while → `429` with a message to show.

GET `/auth/providers` — which social logins are enabled

```
{ "google": true, "google_client_id": "691135919551-....apps.googleusercontent.com", "apple": true }
```

Show the Google and Apple buttons only when they are `true`. Use `google_client_id` as the **serverClientId** for Google sign-in (below).

POST `/auth/google` **NEW: id_token** — Google sign-in

With the `google_sign_in` package, set `serverClientId` to `google_client_id` from `/auth/providers`. Then send the **ID token** you get back:

```
{ "id_token": "<GoogleSignInAuthentication.idToken>" }
```

A Google account with no Voicecon account gets one created automatically, and the response has `user.is_new = true`. For a new user, show "Finish setting up your account on app.voicecon.ai" (onboarding and creating chatbots happen on the web).

POST /auth/apple — Sign in with Apple (iOS)

```
{ "id_token": "<identityToken>", "full_name": "Sara Khan", "nonce": "<raw nonce if you used one>" }
```

Apple sends the user's name only the first time they sign in, so pass `full_name` when you have it. If the user picks "Hide My Email", Apple gives a relay address and a new Voicecon account is created that is separate from their existing one. Tell users to share their real email if they already have an account.

POST /auth/refresh — get new tokens

```
{ "refresh_token": "eyJhbGciOi..." } → same response as login (new access AND refresh token)
```

Use this whenever a request returns `401`. Make only one refresh call at a time (queue other requests while it runs), save **both** new tokens, then retry the original request. If refresh fails with `401`, sign the user out locally.

POST /auth/password/forgot → **POST** /auth/password/reset

```
// 1. send a code by email
{ "email": "sara@example.com" }
// 2. set a new password with that code; returns the login response (user is signed in)
{ "email": "sara@example.com", "code": "123456", "new_password": "....." }
```

GET /users/me — profile for the settings screen

```
{ "id": "...", "email": "sara@example.com", "full_name": "Sara Khan", "avatar_url": null,
  "timezone": "UTC", "language": "en", "is_active": true, ... }
```

Signing out

Important: do **not** call `POST /auth/logout` for a normal "Sign out" in the app. That endpoint signs the account out on **every device, including the web app**. For a normal sign-out:

1. Call `POST /devices/unregister` with this phone's FCM token (section 10).
2. Delete the tokens from secure storage and close the WebSocket.

Use `POST /auth/logout` only for an explicit "Sign out of all devices" option.

After a password reset or "sign out of all devices", the old tokens stop working and this phone stops getting push notifications until the user signs in again and the app registers the device again.

5. Workspaces

An account can belong to several **workspaces** (their own company plus any team they were invited to). Chatbots and conversations belong to a workspace. Every `/chat/*` request must say which workspace it is for with the `X-Organization-Id` header.

GET /workspaces

```
[
  { "id": "173f7f9f-395b-42be-babb-e85ff1faef3f", "name": "Sara Khan's Workspace",
    "slug": "sara-khan", "role": "owner", "is_owner": true, "is_current": true,
    "member_count": 1, "joined_at": "2026-10-07T13:28:49.937507+00:00", "plan_type": "starter" }
]
```

- After sign-in, pick the workspace with `is_current: true` (the one last used on the web). Otherwise pick the first one.
- Remember the choice locally and send it as `X-Organization-Id` on every request.
- If the list has more than one entry, offer a workspace switcher. Switching only changes the header the app sends; no API call is needed.
- Use `role` to decide whether to show the reply box (`viewer` = read-only).
- A push notification includes `organization_id`. If it differs from the current workspace, switch before opening the conversation.

6. Chatbots

GET /chat/chatbots — every chatbot in the workspace

```
{
  "chatbots": [
    {
      "id": "b2057e9d-7c2a-4182-b57c-822dc1c2b066",
      "name": "Online shop",
      "enabled": true,
      "live": true,
      "agent": { "id": "d20a1338-...", "name": "Riley", "is_active": true },
      "config": { "title": "Chat with us", "accent_color": "#4f46e5", ... },
      "session_count": 42,
      "unread_count": 3,
      "unread_sessions": 2,
      "last_activity_at": "2026-10-07T13:28:52.966189+00:00",
      "created_at": "...", "updated_at": "...",
      "public_key": "...", "embed_snippet": "..."
    },
    { "id": "...", "name": "Main website", ... }
  ]
}
```

- Show **all** chatbots. Use `name`; use `config.accent_color` as a colour dot or avatar tint.
- `live: false` means the chatbot is switched off or has no agent. Show it greyed out with "Not live"; its old conversations are still readable.
- `agent` can be `null` (no agent linked).
- `GET /chat/chatbots/{id}` returns one chatbot in the same shape.
- Creating and editing chatbots is done in the web app. The create/update/delete endpoints exist but are not part of the mobile scope.

7. Inbox: conversations, search, filters

A **conversation** (called a "session" in the API) is one website visitor's chat with one chatbot.

GET `/chat/sessions` **NEW** — all conversations, all chatbots

Query param	Meaning
<code>page</code> , <code>page_size</code>	Paging. Default 1 and 20; max page size 100.
<code>chatbot_id</code>	Only this chatbot's conversations.
<code>q</code>	Search text (case-insensitive). Matches any message in the conversation, the page URL where the chat started, and the visitor id.
<code>unread</code>	<code>true</code> = only conversations with unread visitor messages; <code>false</code> = only fully read ones.
<code>mode</code>	<code>human</code> = conversations a team member has taken over; <code>ai</code> = handled by the agent.
<code>from</code> , <code>to</code>	ISO timestamps. <code>from</code> : active since. <code>to</code> : started before.
<code>sort</code>	<code>recent</code> (default) = latest activity first; <code>started</code> = newest conversation first.

```
GET /chat/sessions?page=1&page_size=20&q=delivery&chatbot_id=9b21fba9-...
```

```
{
  "sessions": [
    {
      "id": "d763a8d6-2b68-4e5b-8157-94389d9db666",
      "chatbot_id": "9b21fba9-148c-4dee-967d-700dd004c391",
      "chatbot_name": "Main website",
      "agent_id": "d20a1338-827d-4243-a2a6-0dbf0610ba1b",
      "visitor_id": "v_k2j9x81",
      "visitor_label": "Visitor 9X81",           // show this as the conversation title
      "status": "active",
      "mode": "ai",                             // "ai" | "human"
      "unread_count": 1,
      "message_count": 2,
      "last_message_preview": "Hi! Yes—we can deliver to Lahore. ...",
      "last_message_role": "assistant",         // "user" | "assistant" | "human"
      "source_url": "https://shop.example.com/pricing", // page the chat began on (may be null)
      "started_at": "2026-10-07T13:28:50.522889+00:00",
      "last_activity_at": "2026-10-07T13:28:52.966189+00:00"
    }
  ],
  "total": 1, "page": 1, "page_size": 20, "has_more": false
}
```

Inbox row: title = `visitor_label`, subtitle = `chatbot_name` · `last_message_preview` (prefix "You:" / "AI:" based on `last_message_role`), time = `last_activity_at`, badge = `unread_count`, and a "Team" chip when `mode == "human"`.

Visitors are anonymous website users, so there is no name or email. `visitor_label` is a short stable name made from their browser id.

GET `/chat/chatbots/{chatbot_id}/sessions` — one chatbot's conversations

Same filters and response as above. The default sort here is `started`. You can use either this endpoint or `/chat/sessions?chatbot_id=...`.

GET `/chat/sessions/unread-count` **NEW** — badge numbers

```
{ "unread_count": 5, "unread_sessions": 3 }
```

Use it for the inbox tab badge and the app-icon badge.

8. One conversation: messages, reply, takeover

GET `/chat/sessions/{session_id}` **NEW** — conversation header

The same fields as an inbox row, plus `agent_name` and `user_agent` (the visitor's browser).

GET `/chat/sessions/{session_id}/messages` — the transcript, oldest first

```
{
  "session_id": "d763a8d6-...",
  "mode": "ai",
  "messages": [
    { "id": "8257cb9a-...", "role": "user", "content": "Do you deliver to Lahore?",
      "tool_name": null, "sender": null, "created_at": "2026-10-07T13:28:50.526107+00:00" },
    { "id": "e564d7cd-...", "role": "assistant", "content": "Hi! Yes—we can deliver ...",
      "tool_name": null, "sender": null, "created_at": "2026-10-07T13:28:52.970875+00:00" },
    { "id": "b7d31c25-...", "role": "human", "content": "Yes, we deliver across Pakistan.",
      "tool_name": null, "sender": { "id": "ca34a5b7-...", "name": "Sara Khan" },
      "created_at": "2026-10-07T13:28:53.166546+00:00" }
  ]
}
```

role	Who wrote it	Suggested bubble
user	The website visitor	Left side, grey
assistant	The AI agent	Right side, light colour, small "AI" label
human	A team member (<code>sender.name</code>), from the app or the web	Right side, brand colour, label with the sender's name

`tool_name` is set when the AI used a tool on that turn (for example booked an appointment). You can show it as a small caption.

Only newer messages: add `?after=<created_at of the last message you have>` (URL-encode the `+` as `%2B`). This is useful for polling when the WebSocket is down. Always de-duplicate by message `id`.

POST /chat/sessions/{session_id}/messages **NEW** — reply as a team member

```
{ "content": "Yes, we deliver across Pakistan." } // 1-4000 characters

201 Created
{
  "message": { "id": "b7d31c25-...", "role": "human", "content": "Yes, we deliver across Pakistan.",
    "sender": { "id": "ca34a5b7-...", "name": "Sara Khan" }, "created_at": "...",
    "tool_name": null },
  "session": { ...inbox row..., "mode": "human", "unread_count": 0 }
}
```

- Replying **takes over** the conversation (`mode` becomes `human`). The AI stops answering this visitor, so a person and the AI never both reply.
- Replying also marks the conversation as read.
- The visitor's website chat shows the reply within about 4 seconds, labelled with the sender's first name.
- Show the message straight away (optimistic UI). Replace it with the returned `message` when the request succeeds. The same message also arrives on the WebSocket; drop duplicates by `id` .

POST /chat/sessions/{session_id}/takeover **NEW** — stop the AI without replying yet

POST /chat/sessions/{session_id}/release **NEW** — hand back to the AI

No body. Both return the updated session (`mode` = `human` / `ai`). After release, the AI answers the visitor's next message and can see the team's replies as context. A conversation stays in human mode until someone releases it, so put a clear "Hand back to AI" button in the conversation screen.

POST /chat/sessions/{session_id}/read **NEW** — mark as read

No body; returns the updated session with `unread_count`: 0 . Call it when the conversation screen opens, and again when new visitor messages arrive while the screen is open. Unread counts are shared across the workspace (one team inbox), so a conversation read on the web or by a teammate is read for everyone.

9. Live updates (WebSocket)

WS `wss://api.voicecon.ai/api/v1/chat/ws?token=<access_token>&organization_id=<workspace_id>` **NEW**

Open the socket while the app is in the foreground and close it when the app goes to the background (push takes over then). One socket covers the whole workspace: every chatbot and every conversation.

Messages from the server

Every frame is JSON: `{ "event": "...", "organization_id": "...", "data": { ... } }`

event	data	What to do
ready	— (sent once after connecting)	Connected. Re-fetch the open screen once to catch anything missed while offline.
message.created	<code>{ session_id, chatbot_id, message }</code> . <code>message</code> has the same shape as in the transcript.	If that conversation is open, append it (de-duplicate by <code>id</code>). Fires for visitor, AI and team messages.
session.created	<code>{ session }</code> (inbox row)	A new conversation. Insert it at the top of the inbox.
session.updated	<code>{ session }</code> (inbox row)	Preview, unread count or mode changed. Replace the row and move it to the top if <code>last_activity_at</code> changed.
ping	— (every 25 s)	Keep-alive. Ignore.
pong	—	Answer to your <code>"ping"</code> .

```
{ "event": "message.created",
  "organization_id": "173f7f9f-...",
  "data": {
    "session_id": "d763a8d6-...",
    "chatbot_id": "9b21fba9-...",
    "message": { "id": "8257cb9a-...", "role": "user", "content": "Are you open today?",
                  "tool_name": null, "sender": null, "created_at": "2026-10-07T13:40:02.112+00:00"
                }
  }
}
```

Client rules

- You may send the text `ping`; the server answers with `{"event": "pong"}`. Nothing else needs to be sent.
- Close codes:** `4001` = token invalid or expired (refresh the token, then reconnect); `4003` = no access to that workspace; `4008` = the client fell too far behind (reconnect).
- Reconnect with backoff (1 s, 2 s, 5 s, 10 s, then every 30 s). After each `ready`, re-fetch the inbox and any open conversation so nothing is lost.

- When the access token is refreshed, reconnect with the new token.
- The socket is for speed, not storage. The REST endpoints are always the source of truth.

Fallback: if the socket cannot connect (some corporate networks block WebSockets), poll `GET /chat/sessions/{id}/messages?after=...` every 4–5 seconds while a conversation is open, and `GET /chat/sessions` every 15–30 seconds on the inbox.

10. Push notifications

Push uses **Firebase Cloud Messaging (FCM)** on both Android and iOS (iOS through APNs, configured in Firebase). The server sends a push to every signed-in phone of every workspace member who can read chatbots.

When a push is sent

- A visitor **starts a new conversation** on any chatbot. Title: `New conversation · <chatbot name>`
- A visitor **sends another message** in an existing conversation. Title: `<chatbot name>`
- The body is the visitor's message (up to 180 characters).
- Pushes are **grouped per conversation**: Android notification `tag` and iOS `apns-collapse-id` are the session id, so a chatty visitor updates one notification instead of stacking many.
- AI and team messages do **not** trigger a push (they are not news to the team).

POST `/devices` **NEW** — register this phone

```
{ "token": "<FirebaseMessaging.instance.getToken()>",
  "platform": "android",           // "android" | "ios"
  "device_name": "Pixel 8",       // optional, shown nowhere yet
  "app_version": "1.0.0" }       // optional

→ { "registered": true, "push_enabled": true }
```

- Call it after every successful sign-in, on every app start while signed in, and in `FirebaseMessaging.onTokenRefresh`. It is safe to call repeatedly.
- No `X-Organization-Id` is needed. A phone is registered to the **user** and gets pushes for **all** their workspaces.
- If someone else signs in on the same phone, the token moves to the new user automatically.
- `push_enabled: false` means the server has no Firebase key configured yet; tell the Voicecon team.

POST `/devices/unregister` **NEW** — stop pushes to this phone

```
{ "token": "<same FCM token>" } → { "registered": false }
```

Call it on sign-out (before clearing tokens), and when the user turns notifications off in the app's settings. Call `POST /devices` again to turn them back on.

The push payload

```
{
  "notification": { "title": "New conversation · Main website", "body": "Do you deliver to Lahore?" },
  "data": {
    "type": "chat_message",
    "organization_id": "173f7f9f-395b-42be-babb-e85ff1faef3f",
    "chatbot_id": "9b21fba9-148c-4dee-967d-700dd004c391",
    "session_id": "d763a8d6-2b68-4e5b-8157-94389d9db666",
    "message_id": "8257cb9a-7856-4352-9715-58885aa2853a",
    "new_conversation": "true",          // FCM data values are always strings
    "mode": "ai"
  },
  "android": { "priority": "high", "notification": { "channel_id": "chat_messages", "tag": "<session_id>" } },
  "apns": { "headers": { "apns-collapse-id": "<session_id>" }, "payload": { "aps": { "sound": "default", "thread-id": "<session_id>" } } }
}
```

App setup

- **Android:** create a notification channel with id `chat_messages` (name e.g. "Chat messages", importance high) at app start. Request `POST_NOTIFICATIONS` permission on Android 13+.
- **iOS:** enable Push Notifications and Background Modes → Remote notifications in Xcode. Ask permission with `FirebaseMessaging.instance.requestPermission()`.
- **Tap handling** (`onMessageOpenedApp` and `getInitialMessage`): read `data.organization_id` and switch workspace if needed, then open the conversation `data.session_id` and call `/read`.
- **Foreground:** FCM does not show a banner on Android when the app is open. If the user is already viewing that conversation, show nothing (the socket delivers the message). Otherwise show a local notification (`flutter_local_notifications`) or an in-app banner.
- After a tap or an inbox refresh, update the app-icon badge from `GET /chat/sessions/unread-count`.

11. Putting it together: recommended app flow

App start

1. Load tokens from secure storage. If there are none, show sign-in.
2. GET /users/me . On 401, try /auth/refresh ; if that fails, show sign-in.
3. GET /workspaces → choose the saved workspace, else is_current , else the first one.
4. POST /devices with the FCM token.
5. GET /chat/chatbots and GET /chat/sessions → show the inbox.
6. Open the WebSocket.
7. If the app was opened from a push, go to that conversation.

Conversation screen

1. GET /chat/sessions/{id} + GET /chat/sessions/{id}/messages , then POST .../read .
2. Listen for socket message.created events with this session_id ; append them and call /read again if the new message has role user .
3. Send button → POST .../messages . The header switches to "Team is replying" (mode: human).
4. "Hand back to AI" → POST .../release . "Take over" (when in AI mode) → POST .../takeover .
5. Hide the reply box when the workspace role is viewer .

Search

Debounce typing by about 400 ms, then call GET /chat/sessions?q=<text> (plus the active chatbot filter). Show results with the same row layout, and page through them with has_more .

12. Suggested Flutter packages

Need	Package
HTTP + token refresh interceptor	dio
Secure token storage	flutter_secure_storage
WebSocket	web_socket_channel
Push	firebase_core , firebase_messaging , flutter_local_notifications
Google sign-in	google_sign_in (set serverClientId)
Apple sign-in	sign_in_with_apple
Dates	intl (format), timeago ("2 min ago")
App-icon badge (optional)	app_badge_plus or similar

13. Endpoint quick reference

All paths are under `https://api.voicecon.ai/api/v1`. Auth = Bearer token required. Ws = X-Organization-Id header required.

Method	Path	Auth / Ws	Purpose
POST	<code>/auth/login</code>	— / —	Email + password sign-in
GET	<code>/auth/providers</code>	— / —	Which social logins are on, plus the Google client id
POST	<code>/auth/google</code>	— / —	Google sign-in with <code>id_token</code>
POST	<code>/auth/apple</code>	— / —	Apple sign-in with <code>id_token</code>
POST	<code>/auth/refresh</code>	— / —	New access + refresh token
POST	<code>/auth/password/forgot</code>	— / —	Email a reset code
POST	<code>/auth/password/reset</code>	— / —	Reset with code (signs in)
POST	<code>/auth/logout</code>	✓ / —	Sign out on all devices (only for that option)
GET	<code>/users/me</code>	✓ / —	Profile
GET	<code>/workspaces</code>	✓ / —	Workspaces + role
POST	<code>/devices</code>	✓ / —	Register FCM token
POST	<code>/devices/unregister</code>	✓ / —	Remove FCM token
GET	<code>/chat/chatbots</code>	✓ / ✓	All chatbots + unread counts
GET	<code>/chat/chatbots/{id}</code>	✓ / ✓	One chatbot
GET	<code>/chat/chatbots/{id}/sessions</code>	✓ / ✓	One chatbot's conversations (+ search/filters)
GET	<code>/chat/sessions</code>	✓ / ✓	Inbox across chatbots (+ search/filters)
GET	<code>/chat/sessions/unread-count</code>	✓ / ✓	Badge numbers
GET	<code>/chat/sessions/{id}</code>	✓ / ✓	Conversation header
GET	<code>/chat/sessions/{id}/messages</code>	✓ / ✓	Transcript (<code>?after=</code> for newer only)
POST	<code>/chat/sessions/{id}/messages</code>	✓ / ✓	Team reply (takes over)
POST	<code>/chat/sessions/{id}/takeover</code>	✓ / ✓	Stop the AI
POST	<code>/chat/sessions/{id}/release</code>	✓ / ✓	Hand back to the AI
POST	<code>/chat/sessions/{id}/read</code>	✓ / ✓	Mark read
WS	<code>/chat/ws?token=...&organization_id=...</code>	query	Live events
PUSH	FCM, <code>data.type = "chat_message"</code>	—	Visitor wrote / new conversation

14. Test checklist

Use a staging account that has at least two chatbots installed on a test web page. Play the visitor in a desktop browser and the team member in the app.

1. Sign in with email, Google (Android + iOS) and Apple. A wrong password shows the server message.
2. The chatbot list shows **every** chatbot, including ones that are not live.
3. A visitor message on chatbot A arrives as a push with the app closed; tapping it opens the right conversation.
4. With the app open on the inbox, a new visitor chat appears at the top without refreshing.
5. Search finds a conversation by a word the visitor typed. The chatbot filter narrows results.
6. Reply from the app → the visitor's website chat shows it within about 5 s, and the AI stops answering.
7. "Hand back to AI" → the AI answers the visitor's next message.
8. Opening a conversation clears its unread badge on the phone and on the web.
9. A user in two workspaces can switch between them; a push from the other workspace switches automatically.
10. A viewer-role user can read but cannot reply.
11. Sign out of the app → no more pushes on that phone, and the web app stays signed in.
12. Kill the network for a minute and restore it → the socket reconnects and any missed messages show up.
13. Let the access token expire (or revoke it) → the app refreshes silently, or returns to sign-in.